Day 11: Dictionaries and Sets

1. Key-Value Pairs

A **dictionary** (dict) stores data as **key: value** pairs, like a real dictionary maps a word to its meaning. Instead of remembering that "index 2 is the port," you look things up by a meaningful name.

python

```
server = {
    "name": "Lab-Server-01",
    "ip": "192.168.56.101",
    "port": 22,
    "is_online": True,
}
```

Rules:

- Written with curly braces { }, each pair as key: value, separated by commas.
- **Keys must be unique** and unchangeable (strings, numbers, and tuples work; lists do not).
- **Values can be anything**: numbers, strings, lists, even other dictionaries.
- Since Python 3.7, dictionaries remember insertion order.

Creating dictionaries:

python

```
empty = {}
empty2 = dict()
user = dict(name="alice", role="admin")      # {'name': 'alice', 'role': 'admin'}
pairs = dict([("ssh", 22), ("http", 80)])    # from a list of tuples
```

Reading values:

python

```
print(server["ip"])             # 192.168.56.101
print(server.get("port"))       # 22
print(server.get("os"))         # None (key missing, no crash)
print(server.get("os", "unknown"))   # unknown (custom default)
```

server["os"] on a missing key raises a **KeyError**. Use get() when the key might not exist.

Checking and measuring:

python

```
print("ip" in server)          # True  (checks KEYS, not values)
print("os" not in server)      # True
print(len(server))             # 4
print(type(server))            # <class 'dict'>
```

Useful views:

python

```
print(server.keys())      # dict_keys(['name', 'ip', 'port', 'is_online'])
print(server.values())    # dict_values([...])
print(server.items())     # dict_items([('name', 'Lab-Server-01'), ...])
print(list(server.keys()))   # convert a view to a normal list
```

Nested dictionaries and lists as values:

python

```
hosts = {
    "web": {"ip": "192.168.56.10", "ports": [80, 443]},
    "db":  {"ip": "192.168.56.20", "ports": [3306]},
}


print(hosts["web"]["ip"])          # 192.168.56.10
print(hosts["web"]["ports"][1])    # 443
```

Dictionary vs. list:

Need	**Use**
Ordered items accessed by position	List
Items accessed by a meaningful name	Dictionary
Fixed pair or record that shouldn't change	Tuple

2. Adding and Updating Dictionary Values

Dictionaries are **mutable**, so you can change them freely.

Adding and updating with [ ]

The same syntax does both: if the key exists, the value is **updated**; if not, a new pair is **added**.

python

```
server = {"name": "web-01", "port": 80}


server["os"] = "Ubuntu"        # adds a new key
server["port"] = 8080          # updates an existing key
print(server)
# {'name': 'web-01', 'port': 8080, 'os': 'Ubuntu'}
```

update(): change several at once

```
python
server.update({"port": 443, "is_online": True})
print(server)
```

setdefault(): add only if missing

```
python
server.setdefault("owner", "IT team")    # added
server.setdefault("owner", "someone")    # ignored, key exists
```

Counting with a dictionary (a very common pattern)

```
python
attempts = {}
attempts["bob"] = attempts.get("bob", 0) + 1
attempts["bob"] = attempts.get("bob", 0) + 1
attempts["alice"] = attempts.get("alice", 0) + 1
print(attempts)     # {'bob': 2, 'alice': 1}
```

get(key, 0) returns 0 the first time, so no KeyError occurs.

Removing items

Method	**What it does**
pop(key)	Removes the key and returns its value (KeyError if missing)
pop(key, default)	Same, but returns default instead of crashing
del d[key]	Deletes the pair
popitem()	Removes the most recently added pair
clear()	Empties the dictionary

```
python
```

```
server = {"name": "web-01", "port": 80, "os": "Ubuntu"}


removed = server.pop("os")
print(removed)                  # Ubuntu


server.pop("owner", None)       # safe: no error if missing


del server["port"]
print(server)                   # {'name': 'web-01'}
```

Copying: the same trap as lists

```
python

a = {"x": 1}
b = a              # same dictionary, not a copy
c = a.copy()       # real (shallow) copy
```

Merging (Python 3.9+)

```
python

defaults = {"port": 22, "timeout": 5}
custom = {"port": 2222}
merged = defaults | custom
print(merged)      # {'port': 2222, 'timeout': 5}   (right side wins)
```

3. Looping Through Dictionaries

A plain for loop gives you the **keys.**

```
python

ports = {"ssh": 22, "http": 80, "https": 443}


for name in ports:
    print(name)             # ssh, http, https
```

Loop over values:

```
python

for number in ports.values():
    print(number)           # 22, 80, 443
```

Loop over keys and values together (the most useful form):

python

```
for name, number in ports.items():
    print(f"{name} uses port {number}")
```

Output:

```
ssh uses port 22
http uses port 80
https uses port 443
```

Aligned report:

python

```
for name, number in ports.items():
    print(f"{name:<8}{number:>6}")
```

Searching by value:

python

```
for name, number in ports.items():
    if number == 443:
        print(f"Port 443 is {name}")
        break
```

Filtering into a new dictionary (comprehension):

python

```
privileged = {n: p for n, p in ports.items() if p < 1024}
```

Sorting the output:

python

```
for name in sorted(ports):                    # by key
    print(name, ports[name])


for name, p in sorted(ports.items(), key=lambda item: item[1], reverse=True):
    print(name, p)                            # by value, highest first
```

(lambda is a tiny unnamed function; you'll study functions properly soon. For now, copy the pattern.)

Do not add or remove keys while looping over the same dictionary; Python raises a RuntimeError. Loop over list(d) or build a new dictionary instead.

Looping through nested dictionaries:

python

```
hosts = {
    "web": {"ip": "192.168.56.10", "ports": [80, 443]},
    "db":  {"ip": "192.168.56.20", "ports": [3306]},
}


for host, info in hosts.items():
    print(f"{host} ({info['ip']})")
    for port in info["ports"]:
        print(f"  - port {port}")
```

Use single quotes inside an f-string that uses double quotes.

4. Using Sets to Remove Duplicates

A **set** is an **unordered** collection of **unique** items, written with curly braces but **without** key-value pairs.

python

```
ports = {22, 80, 443, 80, 22}
print(ports)        # {80, 443, 22}  (duplicates gone, order not guaranteed)
```

Creating sets:

python

```
colors = {"red", "green"}
empty = set()           # NOT {}: that makes an empty dictionary!
letters = set("hello")   # {'h', 'e', 'l', 'o'}
```

Key properties:

- Every item appears **once.**
- **No order and no indexing**: my_set[0] raises a TypeError.
- Items must be unchangeable types (no lists inside a set).
- Very fast membership tests with in.

Removing duplicates from a list

python

```
ips = ["10.0.0.5", "10.0.0.7", "10.0.0.5", "10.0.0.9", "10.0.0.7"]


unique_ips = set(ips)
```

print(unique_ips) # duplicates removed, order not guaranteed

print(len(unique_ips)) # 3

Back to a list, sorted:

python

print(sorted(set(ips))) # ['10.0.0.5', '10.0.0.7', '10.0.0.9']

Keeping the original order (since a set loses order):

python

unique_ordered = list(dict.fromkeys(ips))

print(unique_ordered) # ['10.0.0.5', '10.0.0.7', '10.0.0.9']

This works because dictionary keys are unique and remember insertion order.

Adding and removing

python

seen = {"alice", "bob"}

seen.add("carol")

seen.add("bob") # ignored, already there

seen.update(["dave", "erin"])

seen.remove("alice") # KeyError if missing

seen.discard("zed") # no error if missing

print(seen)

Set operations

python

allowed = {22, 80, 443}

open_ports = {22, 80, 3389, 21}

print(open_ports & allowed) # {80, 22} intersection (in both)

print(open_ports | allowed) # union (in either)

print(open_ports - allowed) # {3389, 21} difference (open but NOT allowed)

print(allowed - open_ports) # {443} allowed but not open

print(open_ports ^ allowed) # in one or the other, not both

The difference operation (-) replaces the whole audit loop from Day 10 with one line.

Lists, tuples, dictionaries, sets: quick comparison

Type	Syntax	Ordered	Changeable	Duplicates	Access by
List	[1, 2]	Yes	Yes	Allowed	Position
Tuple	(1, 2)	Yes	No	Allowed	Position
Dictionary	{"a": 1}	Yes (insertion)	Yes	Keys unique	Key
Set	{1, 2}	No	Yes	Not allowed	Membership only

Putting It All Together: Failed Login Analyzer

This analyzes **sample log data stored in the script.** Nothing touches a real system.

Create day11_program.py:

python

```
# Program: Day 11 dictionaries and sets
# Purpose: Summarize failed logins from sample data (practice only)

events = [
    ("bob",   "10.0.0.5",  "failed"),
    ("alice", "10.0.0.7",  "ok"),
    ("bob",   "10.0.0.5",  "failed"),
    ("carol", "10.0.0.9",  "failed"),
    ("bob",   "10.0.0.5",  "failed"),
    ("dave",  "10.0.0.9",  "failed"),
    ("alice", "10.0.0.7",  "failed"),
    ("bob",   "10.0.0.5",  "ok"),
]

THRESHOLD = 3

failures_by_user = {}      # dictionary: username -> count
failing_ips = set()        # set: unique IPs that had a failure
```

```
for user, ip, result in events:
    if result == "failed":
        failures_by_user[user] = failures_by_user.get(user, 0) + 1
        failing_ips.add(ip)

print("=" * 36)
print("  FAILED LOGIN ANALYZER")
print("=" * 36)
print(f"Total events:        {len(events)}")
print(f"Users with failures: {len(failures_by_user)}")
print(f"Unique failing IPs:  {len(failing_ips)}")
print()

print("Failures per user:")
for user, count in sorted(failures_by_user.items(),
                          key=lambda item: item[1], reverse=True):
    flag = "  <-- ALERT" if count >= THRESHOLD else ""
    print(f"  {user:<8}{count:>3}{flag}")

print()
print("IPs with failures:")
for ip in sorted(failing_ips):
    print(f"  {ip}")
```

Output:

```
====================================
  FAILED LOGIN ANALYZER
====================================
Total events:        8
Users with failures: 4
Unique failing IPs:  3
```

Failures per user:

```
bob       3  <-- ALERT
alice     1
carol     1
dave      1
```

IPs with failures:

```
10.0.0.5
10.0.0.7
10.0.0.9
```

Defenders use exactly this idea (count per account or per IP, flag anything over a threshold) to spot brute-force attempts. The set tells you how many *distinct* sources were involved, which separates one noisy attacker from a distributed pattern.

Practice Tasks

1. Create a dictionary for yourself with name, college, and year. Print each value, then add a skills key.
2. Use get() to read a key that doesn't exist, with the default "not set".
3. Build {"ssh": 22, "http": 80, "https": 443} and print each pair as a sentence with .items().
4. Update http to 8080, add ftp: 21, remove ssh safely, and print the dictionary after each step.
5. Given text = "security", count how many times each letter appears using the counting pattern, and print the results.
6. Given words = ["scan", "port", "scan", "log", "port", "scan"], find the most frequent word.
7. Create a dictionary mapping three lab hostnames to lists of open ports, then loop through it and print a report.
8. Given ips = ["10.0.0.5", "10.0.0.7", "10.0.0.5", "10.0.0.9"], remove duplicates three ways: set(), sorted(set()), and dict.fromkeys(). Which one keeps the original order?
9. With allowed = {22, 80, 443} and open_ports = {22, 21, 3389, 80}, print the unexpected ports and the allowed-but-closed ports using set operations.
10. Build a simple username-to-role lookup: ask for a username and print the role, or "Unknown user" if it is not in the dictionary.
11. Extend the Failed Login Analyzer: also count failures per IP in a dictionary and flag any IP with 3 or more failures.
12. Challenge: ask the user to enter several words separated by spaces, then print how many unique words they entered and which word appeared most often.

13. Predict the output, then test:

python

```
d = {"a": 1, "b": 2}
d["c"] = d.get("c", 0) + 5
d["a"] += 10
s = {1, 2, 2, 3}
s.add(3)
s.add(4)
print(d, len(s))
```